

An autonomous loop

A mechanical loop,
three sessions, one state file, and a gate.
No subagents!

Miguel Veloso · miguelveloso.dev

THREE IDEAS

A loop.
Files.
Hard gates.

1 · WHY A LOOP

Fresh context, every task.

Resume is reading a file

Failure stops at one task

2 · WHY FILES

**A prompt you can
review like code.**

Versioned · linkable · reusable

Broken? Edit the file, re-run

3 · WHY HARD GATES

A rule is read
three different ways.

`exit 0` is not

Written **before** the code exists

THE MODEL

The loop is a state machine over artifacts.

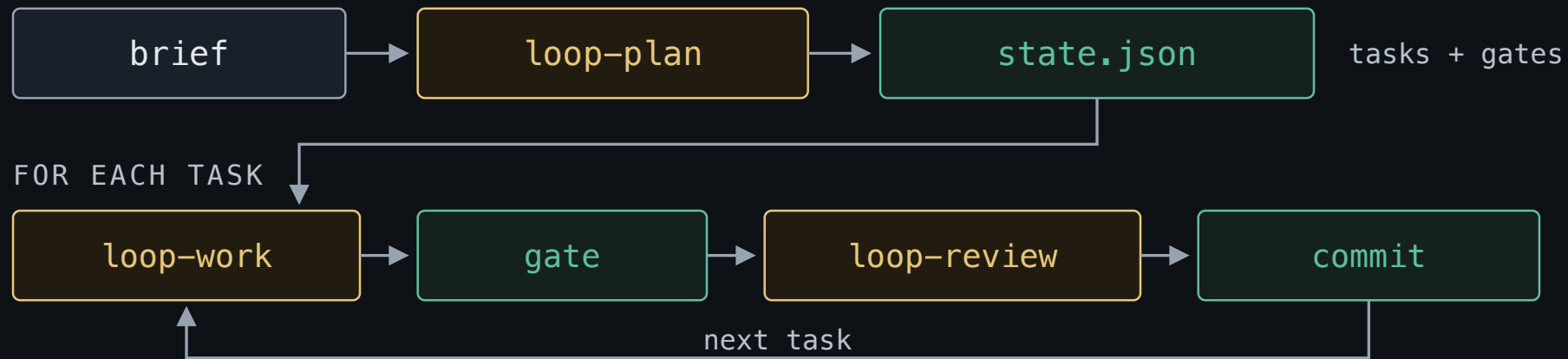
It never reads **your code**

It asks: **did the gate pass?**

So it is **tech agnostic**

The shape of one run

THE LOOP · ONE TASK, START TO COMMIT



Every session is a separate process. Only the driver commits.

claude -p · `sessions` do work and give opinions, never set status

The driver — the only thing that commits

```
claude -p "/loop-plan <brief>"           # once → .loop/state/state.json

while task = next_ready_task(state):      # first pending whose deps are done
    check budgets                         # max iterations · cost ceiling · convergence
    snapshot state.json

    claude -p "/loop-work $task"          # fresh session → tmp/proposal.json
    restore state.json if touched         # the plan is not a session's to edit
    restore any gate file it moved

    for t in done_tasks + [task]:         # THE GATE — every done task, every iteration
        run t.verify                     # a task that stops verifying → pending, attempt burned

    if gate_passed and proposal.outcome == "done":
        claude -p "/loop-review $task"    # fresh session → tmp/verdict.json

apply: done | pending (+1 attempt) | blocked (attempts >= 3)
append journal + telemetry
git commit -m "[loop] $task: $outcome"    # exactly one commit per iteration
```

Three skills, three jobs, no overlap

	loop-plan	loop-work	loop-review
Runs	once per run	once per iteration	once per iteration
Gets	the brief	one task id	the same task id
Reads	brief, <code>CLAUDE.md</code> , knowledge roots	its task, journal, references, the files	the task, the diff , the references
Writes	<code>state.json</code>	code + <code>proposal.json</code>	<code>verdict.json</code> only
Must not	write code	pick another task, touch the plan	edit anything but <code>verdict.json</code>
The point	author every <code>verify</code> before the code exists	do one task and stop	judge what a command cannot check

state.json — the single source of truth

```
{
  "run_id": "0004-runstat-review",
  "brief": "docs/briefs/0004-runstat-review.md",
  "iteration": 6,
  "tasks": [
    {
      "id": "T2",
      "title": "Load reports/NNN-verdict.json in the existing loader",
      "goal": "The verdicts are on disk and nothing can read them...",
      "files": ["src/runstat/loader.py"],
      "depends_on": ["T1"],
      "acceptance": [
        "load_run returns a Run whose verdicts hold one entry per reports/NNN-verdict.json",
        "A verdict file missing any required key raises RunError naming the file"
      ],
      "verify": "uv run python -c \"from runstat.loader import load_run;
        r=load_run('tests/data/run1');
        assert [v.task for v in r.verdicts]==['T1','T2','T2']\"
        && uv run pytest -q",
      "status": "done", "attempts": 0, "notes": ""
    }
  ]
}
```

Every run writes its own evidence

```
.loop/state/runs/<branch>/<run-id>/
├── sessions/
│   ├── 001-work.json      one JSON per session: cost, turns, duration,
│   ├── 002-review.json    permission denials, error flag
├── iterations.jsonl       one line per iteration:
│                           {iteration, task, outcome, attempts, tasks_done}
├── reports/
│   ├── 001-proposal.json  what the work session claimed
│   ├── 001-verdict.json   what the reviewer ruled, criterion by criterion
├── gates/
│   ├── T3.log             every verify command's output
│   ├── 007-T3.fail.log    ...and a pinned copy of each failure
└── loop.log               the driver's own narration
```

Grouped by branch, committed with the iteration, `$HOME` and username masked out.

Two ways to read it

```
$ .loop/run.sh           # at the end of every run
```

— signals —

```
iterations:           11
tasks closed:         11/11
iterations per closed: 1.00
gate failures:        0
review rejections:    0
attempts burned:      0
no-progress streak:   0
estimated spend:      $9.72
```

```
$ uv run runstat summary <run>
```

phase	sessions	cost	turns	wall
work	11	\$ 5.77	148	681s
review	11	\$ 3.95	138	509s
total	22	\$ 9.72	286	1191s

```
$ uv run runstat review <run>
```

```
reviews: 11   passed: 11   failed: 0
criteria ruled: 68   not met: 0
evidence cited: 68/68
coherence: ok
```

The driver prints the signals live and stops on them. **runstat** recomputes the same numbers from the files — and the two agreeing is itself a gate.

Halting cleanly is a success

Faking progress is the only real failure

miguelveloso.dev/blog/an-autonomous-loop